

Robot and Command

This page provides an overview of the `robot` object and the commands used to control the robot, including their roles, components, and execution flow.

Getting Started: Basic Usage

Below is an example of how to use the robot SDK to connect to a robot, power it on, enable servos, and activate the control manager. Detailed explanations of each step can be found in the sections below.

python

```
import rby1_sdk

# Create a robot object
# When accessing via the UPC
robot = rby1_sdk.create_robot_a("192.168.30.1:50051")

# When accessing via the HOTSPOT
# robot = rby1_sdk.create_robot_a("192.168.12.1:50051")

# Connect to the robot
if not robot.connect():
    print("Error: Unable to establish connection to the robot.")
    sys.exit(1)

if not robot.is_connected():
    print("Robot is not connected.")
    exit(1)

# Turn on the robot power
power_dev_name = ".*"
if not robot.is_power_on(power_dev_name):
    rv = robot.power_on(power_dev_name)
    if not rv:
```

```
        print("Failed to power on.")
        exit(1)

# Turn on the robot servos
servo_dev_name = ".*"
if not robot.is_servo_on(servo_dev_name):
    rv = robot.servo_on(servo_dev_name)
    if not rv:
        print("Failed to turn on servos.")
        exit(1)

# Check and reset control manager faults if necessary
control_manager_state = robot.get_control_manager_state()
if (control_manager_state.state == rby1_sdk.ControlManagerState.State.MinorFault or
    control_manager_state.state == rby1_sdk.ControlManagerState.State.MajorFault):
    print("Attempting to reset the fault...")
    if not robot.reset_fault_control_manager():
        print("Error: Unable to reset the fault in the Control Manager.")
        sys.exit(1)
    print("Fault reset successfully.")

# Enable the control manager
if not robot.enable_control_manager():
    print("Error: Failed to enable the Control Manager.")
    sys.exit(1)
```

Robot Object

The `robot` object is responsible for establishing a connection with the robot and managing its state.

- Example: Initialization, connection checks, parameter settings, state updates.

Creating a Robot Object

To create a `robot` object for **RB-Y1 Model A**, use the following method:

- `create_robot_a(address: str) -> Robot_A`
 - Creates a `Robot_A` instance for RB-Y1 Model A.

- **Arguments:**
 - `address` : The IP address of the robot.

For details about the robot model definitions, refer to the **Model** page.

Key Methods

These are the most frequently used methods for controlling the robot. For detailed explanations, refer to the sections below.

- `connect(max_retries: int = 5, timeout_ms: int = 1000) -> bool`
 - Establishes a connection with the robot.
- `power_on(dev_name: str) -> bool`
 - Turns on the power for the specified device.
- `servo_on(dev_name: str) -> bool`
 - Turns on the servo for the specified device.
- `reset_fault_control_manager() -> bool`
 - Resets any fault in the control manager.
- `enable_control_manager(unlimited_mode_enabled: bool = False) -> bool`
 - Enables the control manager, optionally with unlimited mode.

Power and Servo Control

- `power_on(dev_name: str) -> bool`
 - Powers on the specified device.
 - **Arguments:** `dev_name` : Name of the device.
- `power_off(dev_name: str) -> bool`
 - Powers off the specified device.
 - **Arguments:** `dev_name` : Name of the device.
- `servo_on(dev_name: str) -> bool`

- Turns on the servo for the specified device.
- **Arguments:** `dev_name` : Name of the device.
- `is_power_on(dev_name: str) -> bool`
- Checks if the power is on for the specified device.
- `is_servo_on(dev_name: str) -> bool`
- Checks if the servo is on for the specified device.

About `dev_name`

- The `dev_name` argument uses a **regex pattern** to specify the target device(s).
- Examples:
 - To target **all devices**, use `.*` as the pattern.
 - To target specific power devices, such as **48V**, use `48v` as the pattern.
 - For servo operations:
 - To enable all servos for the **right arm**, you can specify each joint explicitly:
`right_arm_0|right_arm_1|right_arm_2|right_arm_3|right_arm_4|right_arm_5|right_arm_6`
 - Alternatively, use the shorter pattern `right_arm.*` to match all joints of the right arm.

Connection Management

- `connect(max_retries: int = 5, timeout_ms: int = 1000) -> bool`
- Establishes a connection to the robot.
- **Arguments:**
 - `max_retries` : Number of retry attempts.
 - `timeout_ms` : Timeout in milliseconds.
- `disconnect() -> None`
- Disconnects the robot.
- `is_connected() -> bool`
- Checks if the robot is connected.

Command Management

- `send_command(builder: RobotCommandBuilder, priority: int = 1) -> Robot_A_CommandHandler`
 - Sends a command to the robot.
 - **Arguments:**
 - `builder` : Command builder object.
 - `priority` : Priority level for the command.
- `create_command_stream(priority: int = 1) -> Robot_A_CommandStreamHandler`
 - Creates a command stream for continuous command sending.
 - **Arguments:**
 - `priority` : Priority level for the stream.
- `cancel_control() -> bool`
 - Cancels the current control.

State and Log Management

- `start_state_update(cb: Callable, rate: float) -> None`
 - Starts periodic updates of the robot's state.
 - **Arguments:**
 - `cb` : Callback function to handle the state updates.
 - `rate` : Update rate in Hz.
- `stop_state_update() -> None`
 - Stops the state updates.
- `start_log_stream(cb: Callable[[list[Log]], None], rate: float) -> None`
 - Starts streaming logs to the callback.
 - **Arguments:**
 - `cb` : Callback to handle log data.
 - `rate` : Log stream rate in Hz.
- `stop_log_stream() -> None`

- Stops log streaming.
- `get_last_log(count: int) -> list[Log]`
- Retrieves the most recent logs.
- **Arguments:**
 - `count` : Number of logs to retrieve.

Control Manager

- `enable_control_manager(unlimited_mode_enabled: bool = False) -> bool`
- Enables the control manager.
- **Arguments:**
 - `unlimited_mode_enabled` : Enables unlimited mode if True.
- Key Points to Understand
 - By default, the control manager **cannot be enabled if any joint is outside its limits**. However, enabling the `unlimited_mode` option allows the control manager to activate **even when joint limits are exceeded**.
-  **Important:**
 - Even with `unlimited_mode` , you **cannot send reference positions outside the joint limits**. This difference must be clearly understood.
- When to Use `unlimited_mode`
 - This feature is particularly useful when the robot's power is turned off, causing the brakes to engage. Over time, the robot's posture may drift, and joints could reach their limits. In such cases, enabling `unlimited_mode` allows you to regain control and correct the posture.
- Alternative Solution
 - If you do not want to use `unlimited_mode` , manually adjust the robot's posture using the physical **joint release switches (brake release buttons)**. Once the posture is corrected, you can activate the control manager in its standard mode.
- `disable_control_manager() -> bool`
- Disables the control manager.

- `get_control_manager_state()` -> `ControlManagerState`
 - Retrieves the state of the control manager.
- `reset_fault_control_manager()` -> `bool`
 - Resets faults in the control manager.

Parameter Management

- `get_parameter(name: str) -> str`
 - Retrieves the value of a parameter.
 - **Arguments:**
 - `name` : Name of the parameter.
- `get_parameter_list() -> list[tuple[str, int]]`
 - Retrieves a list of available parameters.
 - **Return Value:**
 - A list of tuples where:
 - First element: Parameter name.
 - Second element: Parameter type (based on SQLite data types).
- `set_parameter(name: str, value: str, write_db: bool = True) -> bool`
 - Sets the value of a parameter.
 - **Arguments:**
 - `name` : Name of the parameter.
 - `value` : Value to set.
 - `write_db` : Whether to write to the database.
- `reset_parameter_to_default(name: str) -> bool`
 - Resets a specific parameter to its default value stored in the database.
 - **Arguments:**
 - `name` : Name of the parameter.
- `reset_all_parameters_to_default() -> None`
 - Resets all parameters to their default values stored in the database.

- `factory_reset_parameter(name: str) -> bool`
 - Resets a specific parameter to its factory settings (hardcoded defaults).
- **Arguments:**
 - `name` : Name of the parameter.
- `factory_reset_all_parameters() -> None`
 - Resets all parameters to their factory settings (hardcoded defaults).

Notes on Parameter Behavior

- `get_parameter` : Fetches the current value configured in the program, not necessarily the default value.
- **Reset Methods:**
 - `reset_parameter_to_default` / `reset_all_parameters_to_default` :
 - Resets to values stored in the database.
 - `factory_reset_parameter` / `factory_reset_all_parameters` :
 - Resets to factory default values (original hardcoded settings).

Robot Information

- `get_robot_info() -> RobotInfo`
 - Retrieves information about the robot.
-

Commands

Commands define the actions or states to be executed on the robot. This section is recommended to be reviewed alongside the **Command and Control Architecture** for a better understanding.

The methods for sending commands can be broadly categorized into **WholeBodyCommand** and **ComponentBasedCommand**. (There is also a **JogCommand**, but it is not commonly used.) For more details, refer to the **Command and Control Architecture** section mentioned earlier.

Commonly Used Command Types

- **Joint Position Command:** Based on joint positions.
- **Cartesian Command:** Based on Cartesian coordinates.
- **Impedance Control Command:** Based on impedance control.
- **Optimal Control Command:** Based on optimization algorithms.

Joint Position Command Examples

Example 1: Using BodyComponentBasedCommand

python

```
def example_joint_position_command_1(robot):
    print("joint position command example 1")

    # Initialize joint positions
    q_joint_torso = np.zeros(6)
    q_joint_right_arm = np.zeros(7)
    q_joint_left_arm = np.zeros(7)

    # Set specific joint positions
    q_joint_right_arm[1] = -90 * D2R
    q_joint_left_arm[1] = 90 * D2R

    rc = RobotCommandBuilder().set_command(
        ComponentBasedCommandBuilder().set_body_command(
            BodyComponentBasedCommandBuilder()
                .set_torso_command(
                    JointPositionCommandBuilder()
                        .set_minimum_time(MINIMUM_TIME)
                        .set_position(q_joint_torso)
                )
                .set_right_arm_command(
                    JointPositionCommandBuilder()
                        .set_minimum_time(MINIMUM_TIME)
                        .set_position(q_joint_right_arm)
                )
                .set_left_arm_command(
                    JointPositionCommandBuilder()
                        .set_minimum_time(MINIMUM_TIME)
                        .set_position(q_joint_left_arm)
                )
        )
    )
```

```

rv = robot.send_command(rc, 10).get()

if rv.finish_code != RobotCommandFeedback.FinishCode.Ok:
    print("Error: Failed to conduct demo motion.")
    return 1

return 0

```

- **Explanation:**

This example demonstrates how to send a **Joint Position Command** using the **BodyComponentBasedCommand**.

- The torso (6 joints), right arm (7 joints), and left arm (7 joints) are controlled separately.
- Each part's joint angles are defined and sent independently, allowing more modular control of the robot's components.

Example 2: Using BodyCommand

python

```

def example_joint_position_command_2(robot):
    print("joint position command example 2")

    # Define joint positions
    q_joint_waist = np.array([0, 30, -60, 30, 0, 0]) * D2R
    q_joint_right_arm = np.array([-45, -30, 0, -90, 0, 45, 0]) * D2R
    q_joint_left_arm = np.array([-45, 30, 0, -90, 0, 45, 0]) * D2R

    # Combine joint positions
    q = np.concatenate([q_joint_waist, q_joint_right_arm, q_joint_left_arm])

    # Build command
    rc = RobotCommandBuilder().set_command(
        ComponentBasedCommandBuilder().set_body_command(
            BodyCommandBuilder().set_command(
                JointPositionCommandBuilder()
                    .set_position(q)
                    .set_minimum_time(MINIMUM_TIME)
            )
        )
    )

    rv = robot.send_command(rc, 10).get()

```

```

if rv.finish_code != RobotCommandFeedback.FinishCode.Ok:
    print("Error: Failed to conduct demo motion.")
    return 1

return 0

```

- **Explanation:**

This example shows how to send a **Joint Position Command** using the `BodyCommand` .

- All 20 joints of the robot (torso, right arm, and left arm combined) are controlled as a single unit.
- The joint angles for the entire body are defined in one array, allowing unified control over all joints simultaneously.

Key Difference

- **Example 1:** Separates the robot into components (`torso` , `right arm` , `left arm`) using `BodyComponentBasedCommand` for more granular control.
- **Example 2:** Treats the robot as a single body with all 20 joints using `BodyCommand` , simplifying the command structure for whole-body movements.

Cartesian Command Examples

Example 1: Using `BodyComponentBasedCommand`

python

```

def example_cartesian_command_1(robot):
    print("Cartesian command example 1")

    # Initialize transformation matrices
    T_torso = np.eye(4)
    T_right = np.eye(4)
    T_left = np.eye(4)

    # Define transformation matrices
    T_torso[:3, :3] = np.eye(3)
    T_torso[:3, 3] = [0, 0, 1]

    angle = -np.pi / 4
    T_right[:3, :3] = np.array([[np.cos(angle), 0, np.sin(angle)],
                                [0, 1, 0],
                                [-np.sin(angle), 0, np.cos(angle)]])

```

```

T_right[:3, 3] = [0.5, -0.3, 1.0]

T_left[:3, :3] = np.array([[np.cos(angle), 0, np.sin(angle)],
                           [0, 1, 0],
                           [-np.sin(angle), 0, np.cos(angle)]])
T_left[:3, 3] = [0.5, 0.3, 1.0]

# Build command
rc = RobotCommandBuilder().set_command(
    ComponentBasedCommandBuilder().set_body_command(
        BodyComponentBasedCommandBuilder()
        .set_torso_command(
            CartesianCommandBuilder()
            .add_target("base", "link_torso_5", T_torso, LINEAR_VELOCITY_LIMIT,
                       ANGULAR_VELOCITY_LIMIT, ACCELERATION_LIMIT / 2)
            .set_minimum_time(MINIMUM_TIME)
            .set_stop_orientation_tracking_error(STOP_ORIENTATION_TRACKING_ERROR)
            .set_stop_position_tracking_error(STOP_POSITION_TRACKING_ERROR)
        )
        .set_right_arm_command(
            CartesianCommandBuilder()
            .add_target("base", "ee_right", T_right, LINEAR_VELOCITY_LIMIT,
                       ANGULAR_VELOCITY_LIMIT, ACCELERATION_LIMIT / 2)
            .set_minimum_time(MINIMUM_TIME * 3)
            .set_stop_orientation_tracking_error(STOP_ORIENTATION_TRACKING_ERROR)
            .set_stop_position_tracking_error(STOP_POSITION_TRACKING_ERROR)
        )
        .set_left_arm_command(
            CartesianCommandBuilder()
            .add_target("base", "ee_left", T_left, LINEAR_VELOCITY_LIMIT,
                       ANGULAR_VELOCITY_LIMIT, ACCELERATION_LIMIT / 2)
            .set_minimum_time(MINIMUM_TIME * 3)
            .set_stop_orientation_tracking_error(STOP_ORIENTATION_TRACKING_ERROR)
            .set_stop_position_tracking_error(STOP_POSITION_TRACKING_ERROR)
        )
    )
)

rv = robot.send_command(rc, 10).get()

if rv.finish_code != RobotCommandFeedback.FinishCode.Ok:
    print("Error: Failed to conduct demo motion.")
    return 1

```

```
return 0
```

Explanation:

- This example uses `BodyComponentBasedCommandBuilder` to separately define Cartesian commands for the torso, right arm, and left arm.
- Transformation matrices `T_torso` , `T_right` , and `T_left` specify the desired pose of each part relative to a reference frame (e.g., "base").
- `add_target` :
 - Adds a target transformation matrix for a specific component (e.g., torso or end effectors).
 - Arguments:
 - `reference_frame` : The reference frame for the transformation (e.g., "base").
 - `target_frame` : The frame to move (e.g., "link_torso_5" , "ee_right").
 - `T` : 4×4 transformation matrix specifying position and orientation.
 - Velocity and acceleration limits.

Example 2: Using `BodyCommand`

python

```
def example_cartesian_command_2(robot):
    print("Cartesian command example 2")

    # Initialize transformation matrices
    T_torso = np.eye(4)
    T_right = np.eye(4)
    T_left = np.eye(4)

    # Define transformation matrices
    angle = np.pi / 6
    T_torso[:3, :3] = np.array([[np.cos(angle), 0, np.sin(angle)],
                                [0, 1, 0],
                                [-np.sin(angle), 0, np.cos(angle)]])
    T_torso[:3, 3] = [0.1, 0, 1.1]

    angle = -np.pi / 2
    T_right[:3, :3] = np.array([[np.cos(angle), 0, np.sin(angle)],
                                [0, 1, 0],
                                [-np.sin(angle), 0, np.cos(angle)]])
```

```

T_right[:3, 3] = [0.5, -0.4, 1.2]

T_left[:3, :3] = np.array([[np.cos(angle), 0, np.sin(angle)],
                           [0, 1, 0],
                           [-np.sin(angle), 0, np.cos(angle)]])
T_left[:3, 3] = [0.5, 0.4, 1.2]

# Build command
rc = RobotCommandBuilder().set_command(
    ComponentBasedCommandBuilder().set_body_command(
        CartesianCommandBuilder()
            .add_target("base", "link_torso_5", T_torso, LINEAR_VELOCITY_LIMIT, ANGU
                ACCELERATION_LIMIT)
            .add_target("base", "ee_right", T_right, LINEAR_VELOCITY_LIMIT, ANGULAR_VE
            .add_target("base", "ee_left", T_left, LINEAR_VELOCITY_LIMIT, ANGULAR_VE
            .set_stop_position_tracking_error(STOP_POSITION_TRACKING_ERROR)
            .set_stop_orientation_tracking_error(STOP_ORIENTATION_TRACKING_ERROR)
            .set_minimum_time(MINIMUM_TIME)
        )
    )

rv = robot.send_command(rc, 10).get()

if rv.finish_code != RobotCommandFeedback.FinishCode.Ok:
    print("Error: Failed to conduct demo motion.")
    return 1

return 0

```

Explanation:

- This example uses `BodyCommand` to send a unified Cartesian command for the torso and both arms.
- Transformation matrices (`T_torso` , `T_right` , `T_left`) are combined into a single command for whole-body movement.
- `add_target` is used for all components within the same command structure.

Key Points

1. Transformation Matrices (`T`):

- Represent the target pose of a component in a specific reference frame.

- 4×4 matrix combining rotation (`[ :3, :3]`) and translation (`[ :3, 3]`).

2. Difference Between Examples:

- **Example 1:** Modular control for individual components (torso, right arm, left arm).
- **Example 2:** Unified control for the entire robot's movement.

3. CartesianCommandBuilder:

- `add_target` : Adds a specific pose for a component relative to a reference frame.
- `set_minimum_time` : Sets the minimum duration for executing the motion.
- `set_stop_position_tracking_error` : Specifies the allowable position tracking error.
- `set_stop_orientation_tracking_error` : Specifies the allowable orientation tracking error.

Other Commands

For additional command examples, refer to the `demo_motion` source code. It includes examples for:

- **Impedance Control Commands:** Used for force-based control.
 - **Note:** Currently, Impedance Control is only available in **Cartesian Space** and is not supported in **Joint Space**.
- **Optimal Control Commands:** The equations and details for the **Optimal Controller** can be found in the [Command and Control Architecture documentation](#).
- **Mixed Commands:** Combines multiple command types for complex actions.
 - **Note:** Mixed commands do not have a dedicated builder. Instead, multiple command builders are combined when creating commands. This approach is also demonstrated in the `demo_motion` source code.

When executing these motions, it is highly recommended to simulate the robot's actions first in a simulation environment. After verifying the behavior, proceed to run the commands on the real robot. Be cautious, as some motions may involve significant movements.

Always keep the white remote controller in hand during execution on a real robot. The **1st button on the white remote** functions as the emergency stop. Use it immediately if any unexpected situation arises.

Command Execution Flow

Command Creation

- Commands are created using **Command Builders**.

Command Transmission

- Generated commands can be transmitted in two ways:
 - **Once**: Sends the command only once.
 - **Stream**: Sends the command periodically.

Difference Between Once and Stream

- **Once**
 - Sends the command a single time.
 - Suitable for tasks like **via point-based movements** in collaborative robots, where the robot moves from one target point to another.
 - Example: Moving to a single point or performing static motions.
- **Stream**
 - Sends commands periodically.
 - Ideal for tasks requiring trajectory tracking or real-time control, such as **teleoperation**.
 - Example: Continuous motion or real-time trajectory control.

Sending Commands with Once

- Use the `send_command` method from the `robot` object to send a single command.

```
rv = robot.send_command(command, timeout)
```

python

Sending Commands with Stream

- Use the `create_command_stream` method from the `robot` object to create a **stream object**.

- Use this stream object to call `send_command` and transmit commands in a stream.

python

```
stream = robot.create_command_stream()  
rv = stream.send_command(command)
```

[Previous page](#)**M**[Next page](#)**Protos**